

- `print(dict.keys())`
- `print(dict.values())`

You will see the output

- `dict _ keys(['first', 'second'])`
- `dict _ values(['digit', 2018])`

You can also create multiple dictionaries, call several values in one line and see the values and keys of several dictionaries, update or add values to it, access it, as shown here, along with output: <https://digit.in/DictPy>.

Operators

It's better to practice operators in interpreter mode. Simple arithmetic operators include operations like addition, subtraction, multiplication and division and work as expected in most programming languages. In the example below, left number refers to the first argument for the operator whereas right number refers to the second argument for the operator. It is applicable for operators like subtraction, division and modulus. In case of division, Python always returns the value in form of decimals, called as float in Python terminology. If you want to avoid decimals use `//`, as in `>>>8//4` to get 2 instead of 2.0

Other operators are

- `%` (Modulus) – Divides left number by right number and returns remainder.
- `**` (Exponent) – Performs exponential (power) calculation on numbers.

The variables used can be numbers or text. If they are text, encase them in double inverted quotes (“ “).

Relational operators, as the name suggests, compare 2 values and decide the relation between as per the operator used.

`==` If the values of the two variables are equal, then returns value as true.

`!=` If values of two variables are not equal, then returns value as true.

`>` If the value of left variable is greater than the value of right variable, then returns value as true.

`<` If the value of left variable is less than the value of right variable, then returns value as true

`>=` If the value of left variable is greater than or equal to the value of right variable, then returns value as true.

`<=` If the value of left variable is less than or equal to the value of right variable, then returns value as true.

Assignment operators assign a value to a new variable using existing variables